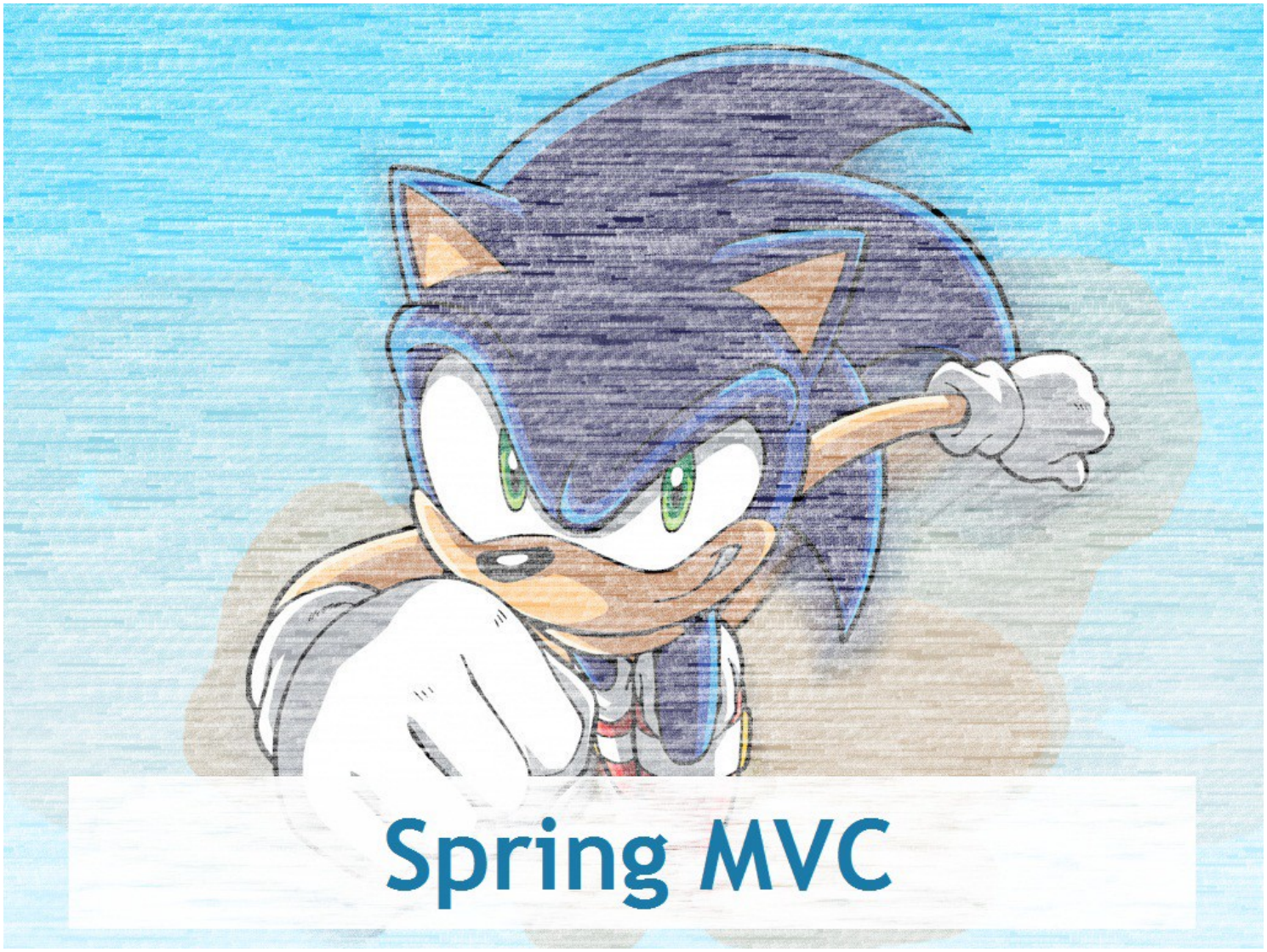
Spring: A Head Start 🔥— Spring MVC (Part 5)

Spring MVC, working with, and validating form data.

Omar Elgabry Follow
Sep 12, 2017 · 9 min read



Spring: A Head Start 🔥 — Spring MVC

Quick Overview

Spring provides Model-View-Controller (MVC) architecture, and components that can be used to develop flexible and loosely coupled web applications. It uses the features of Spring core features like IoC and DI.

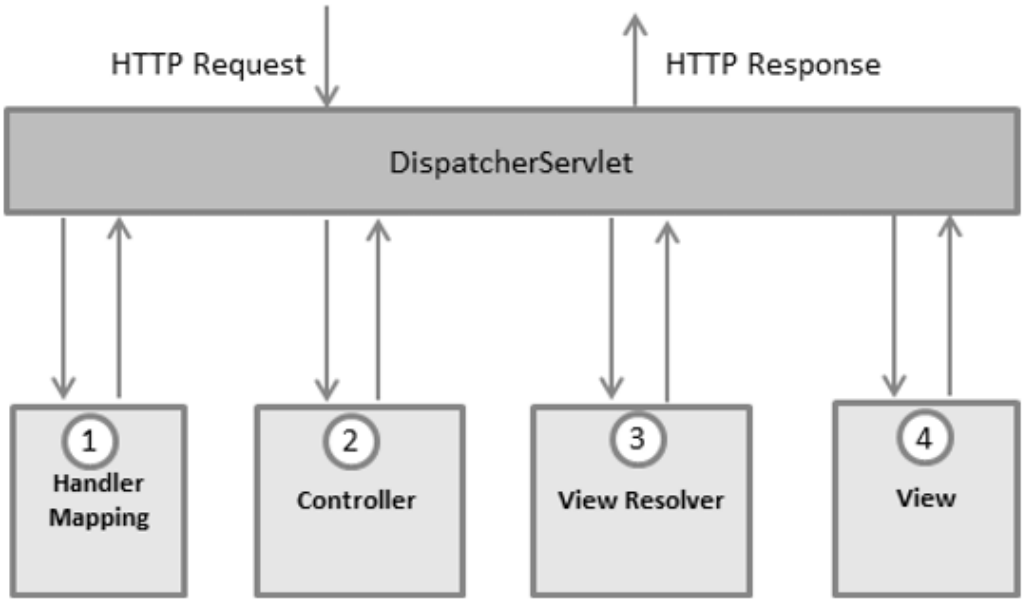
- The **Model** encapsulates the application data and in general they will consist of POJOs.
- The **View** is responsible for rendering the model data and in general it generates HTML output that the client's browser can interpret.
- The **Controller** is responsible for processing user requests and building an appropriate model and passes it to the view for rendering.

The Workflow

The Spring Web MVC framework is designed around the ***DispatcherServlet***.

> 💡*Java Servlet are programs that act as a middle layer between a request coming from a Web browser or HTTP client, and applications on the HTTP server.*

🔍 What happens behind the scene when an HTTP request is sent to the server



Spring MVC Workflow — tutorialspoint

- The *DispatcherServlet* handles all the HTTP requests and responses.

- After receiving an HTTP request, the *DispatcherServlet* consults the *HandlerMapping* to call the appropriate *Controller*.
- The *Controller* takes the request, calls the appropriate method, and builds the model and returns the view page name to the *DispatcherServlet*.
- The *DispatcherServlet* will then take help from the *ViewResolver* to pick up the defined view page for the request.
- Once the view is finalized, the *DispatcherServlet* passes the model data to the view, which is finally rendered on the browser.

> 💡 *All the above-mentioned components (i.e. HandlerMapping, Controller, and ViewResolver) are parts of WebApplicationContext, which is an extension of the ApplicationContext with some extra features necessary for web applications*

Spring MVC Application Components

- A collection of web pages to layout **UI components**.
- A collections of Spring **beans** (controllers, services, etc).
- Spring **configurations** (XML, Annotation, or Java).

Setting Up The Environment 💪

First, you need to create a new "Dynamic Web Project", and add all the jar files to "WEB-INF/lib" folder we added before, or use Maven (as discussed before).

In addition, we'll need to add Java Servlet, JSP, and JSTL. If you're using Maven, there add these dependencies to your "pom.xml" file.

```
<!-- Servlet+JSP+JSTL -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
</dependency>

<dependency>
   <groupId>javax.servlet.jsp</groupId>
   <artifactId>javax.servlet.jsp-api</artifactId>
   <version>2.3.1</version>
```

```
</dependency>

<dependency>
   <groupId>javax.servlet</groupId>
   <artifactId>jstl</artifactId>
    <version>1.2</version>
</dependency>
```

Configurations

[1] Configuring Spring MVC dispatcher servlet(s)

Create an XML file “WEB-INF/web.xml” for defining and configuring Spring MVC dispatcher servlet(s).

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns="http://xmlns.jcp.org/xml/ns/javaee"
 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-a
 id="WebApp_ID" version="3.1">

 <display-name>spring-mvc-demo</display-name>
 <!-- Spring MVC Configs -->

 <!-- Step 1: Configure Spring MVC Dispatcher Servlet -->
 <servlet>
  <servlet-name>MyDispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
   <param-name>contextConfigLocation</param-name>
   <param-value>/WEB-INF/spring-mvc-demo-servlet.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
 </servlet>
</web-app>
```

Then, we setup URL mapping of dispatcher servlet(s) we’ve just defined. This means all requests coming in (since the url-pattern is “/” below) should be handled by dispatcher called “MyDispatcher”.

```
<!-- Step 2: Set up URL mapping of Spring MVC Dispatcher Servlet -->
```

```
<servlet-mapping>
  <servlet-name>MyDispatcher</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>
```

web.xml hosted with ♡ by GitHub view raw

[2] Configuring “MyDispatcher” Dispatcher Servlet

Now, when you make a request, this will be mapped to a specific dispatcher. And so, Spring will load the configuration of that dispatcher from “WEB-INF” folder. In this case, our file is “WEB-INF/spring-mvc-demo-servlet.xml”.

We can add support for features like Spring component scanning, conversions, formatting, validation, configure Spring MVC View Resolver.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:context="http://www.springframework.org/schema/context"
 xmlns:mvc="http://www.springframework.org/schema/mvc"
 xsi:schemaLocation="http://www.springframework.org/schema/beans
     http://www.springframework.org/schema/beans/spring-beans.xsd
     http://www.springframework.org/schema/context
     http://www.springframework.org/schema/context/spring-context.xsd
     http://www.springframework.org/schema/mvc
     http://www.springframework.org/schema/mvc/spring-mvc.xsd">

 <!-- Step 3: Add support for component scanning -->
 <context:component-scan base-package="com.springmvcdemo" />

 <!-- Step 4: Add support for conversion, formatting and validation support -->
 <mvc:annotation-driven/>

 <!-- Step 5: Define Spring MVC view resolver -->
 <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/view/" />
  <property name="suffix" value=".jsp" />
 </bean>
</beans>
```

spring-mvc-demo-servlet.xml hosted with ♡ by GitHub view raw

> 💡 *The “prefix” and “suffix” will be used to render the view page.*

> 📢 *We need to create "view" folder inside the "WEB-INF" folder.*
>
> 📢 *I'm assuming you've created a package called "com.springmvcdemo" where all of your Java classes live. The package name has to be aligned with "Step 3" above.*

— How to add CSS, JavaScript and Images?

Any static resource is processed as a URL mapping in Spring MVC.

You can configure references to static resources by adding the following entry to the configuration file of the Dispatcher Servlet.

```
<!-- A handler for serving static resources -->
<mvc:resources mapping="/assets/**" location="/assets/" />
```

> *The double asterisk "**" means to look also for the subdirectory.*

In your view pages, we can access the static files using one of the following:

```
<link rel="stylesheet" type="text/css"
href="${pageContext.request.contextPath}/assets/css/main.css">
```

Or, …

```
<!-- Include the core JSTL tags (will be discussed later) -->
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>

<!-- ... -->

<link rel="stylesheet" type="text/css"  href="<c:url
value="/assets/css/main.css" />">
```

Spring MVC Application

Now we understand the concepts, and configured our Spring MVC application. Let's see how can we apply these concepts in action … 🏃

- The *DispatcherServlet* delegates the request to the appropriate controller to handle the request.
- The `@Controller` annotation indicates that a particular class serves as the role of a controller.
- The `@RequestMapping` annotation is used to map a URL to either an entire class or a particular controller method.

[1] Create controller class using @Controller annotation

The `@Controller` annotation inherits from `@Component` annotation. So, It's a special `@Component` used in MVC web application, and so, it will be scanned by Spring IoC as a component.

```
package com.springmvcdemo;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/")
public class WelcomeController {
}
```

WelcomeController.java hosted with ♡ by GitHub view raw

So, now, this controller will handle all the requests for root path URL of our *MyDispatcher* we've created.

[2] Define & Map Controller Methods

We can pass different parameters like request, session, model objects, and return different types of objects (as you'll see later) from controller method.

All request mapping on methods are relative to the controller mapping.

```
package com.springmvcdemo;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
@RequestMapping("/")
public class WelcomeController {

  @RequestMapping(value = "/", method = RequestMethod.GET)
  public String welcome(Model model){ /*...*/ }

  @RequestMapping(value = "/profile", method = RequestMethod.GET)
  public String profile(Model model){ /*...*/ }
}
```

WelcomeController.java hosted with ♡ by GitHub view raw

So, now, the `welcome()` method will handle root path of this controller.

By default, It handles all types of HTTP request (GET, POST, etc) unless explicitly defined (i.e. using `method` in `@RequestMapping`).

[3] Adding an attribute (data) to the model

We mean by **model** here is an object that we can use to pass data around the controller methods and views.

These data is called **model attribute(s)**, they can be java beans that carry form data for data binding.

> *We'll discuss HTML form data and data binding later.*

So, we can pass a model attribute from a controller method to a view page, and then from the view page back to a controller method (i.e. when form is submitted).

The model object can be passed as a parameter to controller method so that we can add attributes, and these attributes can be accessed by the view to display the final result.

```
package com.springmvcdemo;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
```

```
@Controller
@RequestMapping("/")
public class WelcomeController {

 @RequestMapping(value = "/", method = RequestMethod.GET)
 public String welcome(Model model){
   model.addAttribute("message", "Hello Spring MVC Framework!");
   /*...*/
 }
}
```

WelcomeController.java hosted with ♡ by GitHub view raw

[4] The Controller Method Returned Value

We can return a view page name, an object, or not return anything at all.

In case of returning a view page name, Spring will load the view page from using this format: **`prefix`** `+ view page name +` **`suffix`** (defined previously in "*spring-mvc-demo-servlet.xml*" file) to be rendered and displayed.

```
package com.springmvcdemo;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
@RequestMapping("/")
public class WelcomeController {

 @RequestMapping(value = "/", method = RequestMethod.GET)
 public String welcome(Model model){
   model.addAttribute("message", "Hello Spring MVC Framework!");
   return "welcome-page";
 }
}
```

WelcomeController.java hosted with ♡ by GitHub view raw

[5] Create the view pages

These are mostly the JSP pages (but you can have HTML, PDF, Excel worksheets, XML, etc). In JSP pages, you write your HTML, and use the reusable UI components (like tags) in Spring.

In our case, we need to create “welcome-page.jsp” file.

> ⚠ *The view pages must be located in the location we’ve defined earlier in* “spring-mvc-demo-servlet.xml” *file.*

```
<html>
 <head>
   <title>Spring MVC Demo</title>
 </head>
 <body>
   <h2>${message}</h2>
 </body>
</html>
```

welcome-page.jsp hosted with ♡ by GitHub view raw

> *The* `${message}` *is the attribute which we assigned in* `welcome()` *method.*

Testing Our Project

To test the project hit the URL → *http://localhost:8080/[NameOfTheProject]/*

Working With HTML Form Data

In this section example, we’ll explain how to work with HTML form data; read the submitted data and display the results.

Before we start, let’s say we have a Java class *User* acts as a model in our MVC Spring application. So, we can add new user, and display user’s data.

```
package com.springmvcdemo;

public class User {
  private String name;
  private Integer id;
```

```
    public User(){
    }

    public void setName(String name) {
      this.name = name;
    }

    public String getName() {
      return name;
    }

    public void setId(Integer id) {
      this.id = id;
    }

    public Integer getId() {
      return id;
    }
  }
```

User.java hosted with ♡ by GitHub view raw

[1] Display HTML Form

For the first time, we will display an empty form, right?. So, we can create a model attribute (let's say called *user*), and assign it to an empty user object.

So, initially, an empty form will be displayed for adding new user.

```
@RequestMapping("/")
public String newUser(Model model){
  model.addAttribute("user", new User());
  return "userForm";
}
```

UserController.java hosted with ♡ by GitHub view raw

[2] Set-Up The Form

In Spring, we have reusable UI components we can use to build our HTML such as the form tags. The form tags allows for data binding; automatically set and retrieve data from java beans (model attributes).

All what we need to do is to reference the Standard Tag Library (JSTL) in our JSP pages. They are classified into 5 groups: *Core, Formatting, SQL, XML, and JSTL Functions*.

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<html>
 <head>
   <title>Spring MVC Demo</title>
 </head>
 <body>
  <h2>User Data</h2>
   <form:form action="addUser" modelAttribute="user" method="POST">
     <form:label path="name">Name</form:label>
     <form:input path="name" /> <br><br>
     <form:label path="id">Id</form:label>
     <form:input path="id" />

     <input type="submit" value="Submit"/>
    </form:form>
 </body>
</html>
```

userForm.jsp hosted with ♡ by GitHub view raw

> 🔍 ***Behind The Scene …***
>
> *When the form is loaded, Spring will populate the user objects' field values using getters. If it's null, the form input field will be empty. When the form is submitted, Spring will assign the user objects' field values using setters.*

[4] Reading Form Data

— HttpServletRequest

We can pass and use the *HttpServletRequest* object in a controller method to get access to all request parameters (i.e. GET request).

```
@RequestMapping("/user")
public String showUser(HttpServletRequest request){
  int id = Integer.parseInt(request.getParameter("id"));
  /*...*/
}
```

— RequestParam Annotation

We can also read form data using `@RequestParam` annotation. This is a different way for reading form data in the request parameters.

Here, we bind a variable with a value fetched from `@RequestParam` annotation.

```
@RequestMapping("/user")
public String showUser(@RequestParam("id") int id) {/*...*/}
```

— Accessing request parameter values in view pages

There are set of object made available in JSP pages such as *request*, *response*, *param*, *paramValues*, *cookie*, etc.

The *param* object maps a request parameter name to a single value, while the *paramValues* maps a request parameter name to an array of values.

```
<html>
 <head>
   <title>Spring MVC Demo</title>
 </head>
 <body>
   <h2>${param.id}</h2>
 </body>
</html>
```

— RequestBody Annotation

If the data is passed in the request body (i.e. POST request), then, we can access the data using `@RequestBody` instead.

```
@RequestMapping(value = "/addUser", method = RequestMethod.POST)
public String addUser(@RequestBody User user){/*...*/}
```

> 💡*Similarly, a controller method with* `@ResponseBody` *annotation indicates that the returned value should be bound to the web response body.*

— ModelAttribute Annotation

When the form is submitted (let's say for adding a new user), we can **bind** the user object with form data submitted. *How?* The model attribute that come with `@ModelAttribute` annotation is bind-ed to the data submitted.

So, we can bind the user object with the model attribute (call it *user* for example), and access all the data submitted using the user object.

```
@RequestMapping(value = "/addUser", method = RequestMethod.POST)
public String addUser(@ModelAttribute("user") User user) {
  String name = user.getName();
  int id      = user.getId();
  /*...*/
}
```

UserController.java hosted with ♡ by GitHub view raw

> 💡 *The* `@ModelAttribute` *annotation makes the model attribute* user *available to view pages to access and display it.*
>
> 📣 *This is the method we will be using in our example.*

[5] Display Data Submitted

Now, we can use the model attribute *user* to display it's data in the view page.

```
@RequestMapping(value = "/addUser", method = RequestMethod.POST)
public String addUser(@ModelAttribute("user") User user) {
  return "result";
}
```

UserController.java hosted with ♡ by GitHub view raw

In the *result.jsp* view page, we can access and display model attributes.

```
<html>
 <head>
   <title>Spring MVC Demo</title>
 </head>
 <body>
  <h2>User Data</h2>
```

```
    <table>
      <tr>
         <td>Name</td>
         <td>${user.name}</td>
      </tr>
      <tr>
         <td>ID</td>
         <td>${user.id}</td>
      </tr>
    </table>
  </body>
 </html>
```

result.jsp hosted with ♡ by GitHub view raw

Form Data Validation

Spring 4 supports the Bean Validation API. Hibernate Validator is an implementation for the Java Beans Validation API.

It provides a set of validation annotations we can use to validate the data, such as `@NotNull`, `@Max`, `@Min`, `@Size`, `@Pattern`, `@Future` & `@Past`.

To add and use hibernate validator. Either download JAR files from and add them to our "lib" folder in the project, or declare the dependency in your Maven "pom.xml" file.

Form Data Validation — The Process

We're going to use the example above of adding a new user further an apply validation rules on the submitted data.

[1] Add Validation Rules To User Class

For each filed of user object, we'll specify the validation rule(s).

— The **name** is required, has min length of 2 and a max of 20. It must only contain alpha numeric characters with spaces.

```
package com.springmvcdemo;

import com.springmvc.validation.constraints.Pattern;
```

```
import javax.validation.constraints.Pattern;
import javax.validation.constraints.Size;

import org.hibernate.validator.constraints.NotBlank;

public class User {

  // required, max & min length, alphaNumWithSpaces
  @NotBlank(message="name can't left blank")
  @Size(min=2, max=20, message="name must have min length of 2, and max of 20")
  @Pattern(regexp = "^[\\p{Alnum}\\s]+$", message="name can only contain alpha numeric character
  private String name;

  /* ... */
}
```

User.java hosted with ♡ by GitHub view raw

— The **password** is required, can't be empty, min length of 8, and must contain at least one digit, one lower char and one upper char, one special char, and doesn't contain spaces.

```
package com.springmvcdemo;

import javax.validation.constraints.Pattern;
import javax.validation.constraints.Size;

import org.hibernate.validator.constraints.NotBlank;

public class User {

  // required, strong password, min length
  @NotBlank(message="password can't left blank")
  @Size(min=8, message="password can't be less than 8 chars")
  @Pattern(regexp="^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&+=])(?=\\S+$).+$", message="pas
  private String password;

  /* ... */
}
```

User.java hosted with ♡ by GitHub view raw

— The **email** is required, and has a valid email format.

```java
package com.springmvcdemo;

import org.hibernate.validator.constraints.NotBlank;
import org.hibernate.validator.constraints.Email;

public class User {

  // required, a valid email format
  @NotBlank(message="email can’t left blank")
  @Email(message="please enter a valid email", regexp=".+@.+\\..+")
  private String email;

  /* ... */
}
```

User.java hosted with ♡ by GitHub view raw

— The **date of birth** can’t be null, and in the past.

```java
package com.springmvcdemo;

import java.util.Date;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Past;

public class User {

  // required, date in the past
  @NotNull(message="birth date can't left blank")
  @Past
  private Date birth;

  /* ... */
}
```

User.java hosted with ♡ by GitHub view raw

— The **age** is required, min is 15, and max is 100.

```java
package com.springmvcdemo;

import javax.validation.constraints.Max;
import javax.validation.constraints.Min;
```

```
import javax.validation.constraints.NotNull;

public class User {

  // required, min is 15 and max is 100
  @NotNull(message="age is required")
  @Min(value=5, message="age can't be less than 5")
  @Max(value=100, message="age can't be greater than 100")
  private Integer age;

  /* ... */
}
```

User.java hosted with ♡ by GitHub view raw

— The **height** has value between 50 and 300.

```
package com.springmvcdemo;

import org.hibernate.validator.constraints.Range;

public class User {

  // value between 50 and 300
  @Range(min=50, max=300, message="height has to be within 50 and 300")
  private Integer height;

  /* ... */
}
```

User.java hosted with ♡ by GitHub view raw

— The array of objects **hobbies** can't be empty; there must be at least one.

```
package com.springmvcdemo;

import org.hibernate.validator.constraints.NotEmpty;

public class User {

  // can't be empty; there must be at least one.
  @NotEmpty(message="you must choose at least one")
  private String[] hobbies;

```

```
  /* ... */
}
```

User.java hosted with ♡ by GitHub view raw

📣 One common question here, *"What's the difference between* `@NotNull`, `@NotEmpty`, *and* `@NotBlank` *?"*

- `@NotNull` : Checks if the value is not null, disregarding the content (it can be empty). *Used with dates, collections, map, array, etc.*
- `@NotEmpty` : Checks if the value is not null nor empty, allowing empty spaces but size has to be greater than zero. *Used with strings, collections, map, array etc.*
- `@NotBlank` : Checks if the value is not null nor empty, and the **trimmed** length is greater than zero. *Used with strings.*

[2] Display Error Message In HTML Form

We'll use `form:errors` tag to display the validation error messages (if exist). In addition we can apply a CSS class for displaying errors.

```
<form:form action="addUser" modelAttribute="user" method="POST">
  <form:label path="name">Name</form:label>
  <form:input path="name" />
  <form:errors path="name" cssClass="error" /><br><br>

  <!-- ... -->
  <input type="submit" value="Submit"/>

</form:form>
```

userForm.jsp hosted with ♡ by GitHub view raw

[3] Perform Validation Rules In The Controller

The `@Valid` annotation performs validation rules on user object (we defined earlier in user class). The results of the validation will be placed in the *BindingResult* object.

```
package com.springmvcdemo;

import javax.validation.Valid;
```

```java

import org.springframework.stereotype.Controller;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.ModelAttribute;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
@RequestMapping("/")
public class WelcomeController {

  /* ... */

  @RequestMapping(value = "/addUser", method = RequestMethod.POST)
  public String addUser(@Valid @ModelAttribute("user") User user, BindingResult results) {
    if(results.hasErrors())     return "userForm";
    else                        return "result";
  }
}
```

UserController.java hosted with ♡ by GitHub view raw

> *If error exists, it will be displayed as a `span` replacing the `form:errors` tag.*

```
<span id="name.errors" class="error">name must have min length of 2,
and max of 20</span>
```

Custom Validation Rules

The annotation validation rules we have used are pre-defined. In some cases, you need to define your own custom logic.

[1] Create a Custom Annotation For Validation Rule

```java
package com.springmvcdemo;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import javax.validation.Constraint;
```

```
import javax.validation.Payload;

// Below are annotations to describe the annotation to be created
// VisaCodeConstraintValidator: A helper class that has the logic code for validation (see next
@Constraint(validatedBy = VisaCodeConstraintValidator.class)

// Where we can apply this annotation?
// For example, on a method and field
@Target( {ElementType.METHOD, ElementType.FIELD} )

// For how long should we retain(store) it?
// RetentionPolicy.RUNTIME Keep this annotation in the compiled java bytecode and use it during
@Retention(RetentionPolicy.RUNTIME)
public @interface VisaCode {

  // define the parameters
  public String value()  default "VJ";
  public String message() default "must start with a country code and has length >= 5";

  // grouping validation constrains together
  public Class<?>[] groups() default {};

  // addition information about validation error
  public Class<? extends Payload>[] payload() default {};
}
```

[2] Create A Custom Validation Rule Code (Helper Class)

```
package com.springmvcdemo;

import javax.validation.ConstraintValidator;
import javax.validation.ConstraintValidatorContext;

// <The annotation, Type of data we are validating against>
public class VisaCodeConstraintValidator implements ConstraintValidator<VisaCode, String> {

  // developer supplied visa code to match against the user value
  private String requiredVisaCode;

  @Override
  public void initialize(VisaCode code) {
    requiredVisaCode = code.value();
  }
```

```
  }

  @Override
  public boolean isValid(String userVisaCode, ConstraintValidatorContext arg1) {
    // You need to check first if the value is null
    // Because if the user didn't enter any value, this code will throw an exception
    return (userVisaCode != null && userVisaCode.startsWith(requiredVisaCode) && userVisaCode.le
  }
}
```

[3] Use The Custom Validation Rule

```
@VisaCode(value="IND", message="visa code must start with IND and of
length >= 5")
private String visaCode;
```

Thank you for reading! If you enjoyed it, please clap 👏 for it.

Java Spring Framework Programming Software Development Other

About Help Legal

Get the Medium app

Download on the App Store GET IT ON Google Play